

Homomorphic Encryption

Bill Buchanan

Outline

September 4, 2024



Chapter 1

Introduction

This document is split into 12 main chapters:

1. Background.
2. State of the Art Methods.
3. Examine secure data sharing.
4. Cloud-based operations.
5. Secure multi-party computation.
6. Machine learning applications.
7. Modern implementations of HE.
8. Quantum resilience.
9. Hardware requirements.
10. Data fragmentation.
11. Side Channel Analysis.
12. Additional research area.

The delivery of milestones are:

1. Milestone 1 (Sep 2024): Delivery of chapters 1 and 2.
2. Milestone 2 (4 Oct 2024): Delivery of chapters 4, 5, 6 and 7.
3. Milestone 3 (6 Dec 2024): Delivery of the rest of the chapters.

Chapter 2

Background

2.1 Introduction

This chapter outlines some of the fundamental principles involved with homomorphic encryption, including the usage of polynomials and learning with errors (LWE). With homomorphic encryption we can take plaintext of A and B and produce:

$$Enc_k(A \circ B) = Enc_k(A) \circ Enc_k(B) \quad (2.1)$$

and where $\circ$ can be any arithmetic operator such as add, subtract, multiply and divide. With symmetric key encryption, we use the same key to decrypt as we do to encrypt.

2.2 Partial homomorphic

With partial homomorphic encryption (PHE), we can implement some form of arithmetic operation in a homomorphic way. These methods include Paillier [1], ElGamal and a modified RSA version.

2.2.1 Paillier

The Paillier cryptosystem is a partial homomorphic encryption (PHE) method and can perform addition, subtraction and scalar multiply. Thus we get:

$$Enc_k(A + B) = Enc_k(A) + Enc_k(B) \quad (2.2)$$

$$Enc_k(A - B) = Enc_k(A) - Enc_k(B) \quad (2.3)$$

$$Enc_k(A.B) = A.Enc_k(B) \quad (2.4)$$

If we take two values: m_1 and m_2 , we get two encrypted values of $Enc(m_1)$ and $Enc(m_2)$. We can then multiply the two cipher values to get $Enc(m_1 + m_2)$. We can then decrypt to get $m_1 + m_2$. Along with this, we can also subtract to $Enc(m_1 - m_2)$. This is achieved by taking the inverse modulus of $Enc(m_2)$ and multiplying it with $Enc(m_1)$. Finally, we can perform a scalar multiply to get $Enc(m_1 \cdot m_2)$ and which is generated from $Enc(m_1)^{m_2}$.

First we select two large prime numbers (p and q) and compute:

$$N = pq \quad (2.5)$$

$$PHI = (p - 1)(q - 1) \quad (2.6)$$

$$\lambda = \text{lcm}(p - 1, q - 1) \quad (2.7)$$

and where lcm is the least common multiple. We then select a random integer g for:

$$g \in \mathbb{Z}_{nN^2}^* \quad (2.8)$$

We must make sure that n divides the order of g by checking the following:

$$\mu = (L(g^\lambda \pmod{n}^2))^{-1} \pmod{N} \quad (2.9)$$

and where L is defined as $L(x) = \frac{x-1}{N}$. The public key is (N, g) and the private key is (λ, μ) .

To encrypt a message (M), we select a random r value and compute the ciphertext of:

$$c = g^m \cdot r^N \pmod{N^2} \quad (2.10)$$

and then to decrypt:

$$m = L(c^\lambda \pmod{N}^2) \cdot \mu \pmod{N} \quad (2.11)$$

For adding and scalar multiplying, we can take two ciphers (C_1 and C_2), and get:

$$C_1 = g^{m_1} \cdot r_1^N \pmod{N^2} \quad (2.12)$$

$$C_2 = g^{m_2} \cdot r_2^N \pmod{N^2} \quad (2.13)$$

If we now multiply them, we get:

$$C_1 \cdot C_2 = g^{m_1} \cdot r_1^N \cdot g^{m_2} \cdot r_2^N \pmod{N^2} \quad (2.14)$$

$$C_1 \cdot C_2 = g^{m_1+m_2} \cdot r_1^N \cdot r_2^N \pmod{N^2} \quad (2.15)$$

And when adding two values requires a multiplication of the ciphers. Examples of using Paillier are defined at [2].

2.2.2 ElGamal - Multiply and Divide

With ElGamal public key encryption, we have a public key of (Y, g, p) and a private key of (x) . The cipher has two elements $(a$ and $b)$. With this, Bob selects a private key of x and then calculates $Y (g^x \pmod p)$ for his public key. We can use it for partial homomorphic encryption, and where we can multiply the ciphered values. With this, we encrypt two integers and then multiply the ciphered values, and then decrypt the result [3].

Initially, Bob creates his public key by selecting a g value and a prime number (p) and then selecting a private key (x) . He then computes Y which is:

$$Y = g^x \pmod p \quad (2.16)$$

His public key is (Y, g, p) and he will send this to Alice. Alice then creates a message (M) and selects a random value k . She then computes a and b :

$$a = g^k \pmod p \quad (2.17)$$

$$b = y^k M \pmod p \quad (2.18)$$

Bob then receives these and decrypts with:

$$M = \frac{b}{a^x} \pmod p \quad (2.19)$$

This works because:

$$\frac{b}{a^x} \pmod p = \frac{y^k M}{(g^k)^x} \pmod p \quad (2.20)$$

$$= \frac{(g^x)^k M}{(g^k)^x} \pmod p \quad (2.21)$$

$$= \frac{g^{xk} M}{g^{xk}} \pmod p \quad (2.22)$$

$$= M \quad (2.23)$$

2.2.3 RSA

RSA is a partially homomorphic encryption method, and where we can add and multiply encrypted values. So with RSA we cipher with [4]:

$$Cipher = M^e \pmod N \quad (2.24)$$

and where e is the encryption key, and N is the multiplication of two prime numbers (p and q). The difficulty in this is factorizing N to get two prime numbers. To decrypt, we find the decryption key (d) and then perform:

$$M = Cipher^d \pmod{N} \quad (2.25)$$

So if we take two values (a and b), then we can cipher them as:

$$Cipher_1 = a^e \pmod{N} \quad (2.26)$$

$$Cipher_2 = b^e \pmod{N} \quad (2.27)$$

If we multiply the ciphers we get:

$$Cipher_3 = Cipher_1 \times Cipher_2 \quad (2.28)$$

$$= a^e \pmod{N} \times b^e \pmod{N} \quad (2.29)$$

$$= a^e \times b^e \pmod{N} \quad (2.30)$$

This is then:

$$Cipher_3 = (ab)^e \pmod{N} \quad (2.31)$$

When we can then decrypt and determine the value of a times b .

$$ab = (Cipher_3)^d \pmod{N} \quad (2.32)$$

2.3 Polynomials

With lattice methods, we have multi-dimensional spaces, and where to plot our points within there. Most of the time, we think in one dimension (x), or two dimensions (x, y), or three dimensions (x, y , and z), but in a lattice, we may have thousands of dimensions. Each point we have then fitted into a lattice of these points. To make a hard problem to solve, we add a small amount of error to the point. It is then extremely difficult to find the nearest point to the point with the error.

With lattices, we use polynomials to represent our multi-dimensional spaces. In general, a polynomial can be represented by a number of coefficients ($a_n \dots a_0$) and polynomial powers:

$$f(x) = a_n x^n + a_{n-1} x^{n-1} + \dots a_1 x + a_0 \quad (2.33)$$

Within a lattice, we can have a point at $(9, 5, 16)$, and then represent it with a quadratic equation of:

$$16x + 5x + 9 \quad (2.34)$$

This could be represented by a polynomial of $[16, 9, 5]$. If we have another polynomial of $2x^2 + x + 7$, and multiply them together, we get:

$$f = (16x^2 + 5x + 9)(2x^2 + x + 7) = 32x^4 + 26x^3 + 135x^2 + 44x + 63 \quad (2.35)$$

In Python, we can determine this with:

```
from numpy.polynomial
import Polynomial
import numpy as np
f=Polynomial([16,9,5])g=Polynomial([2,1,7])
r = f*g
print(r.coef)
print(np.poly1d(r.coef))
```

And a sample run gives:

```
[ 32.  26. 135.  44.  63.]
   4      3      2
32 x + 26 x + 135 x + 44 x + 63
```

2.3.1 Mod polynomials

With polynomials, for an order of five, we have the form:

$$ax^5 + bx^4 + cx^3 + dx^2 + ex + f \quad (2.36)$$

This can then represent this as an integer array of:

$$[a, b, c, d, e, f] \quad (2.37)$$

In Python, the polynomial of $5x^4 + 4x^3 + x^2 + 11x + 10$ is implemented and printed as:

```
import numpy as np
A=[5,4,1,11,10]
print ("A:")
print(np.poly1d(A))
```

This gives:

```
A:
   4      3      2
5 x + 4 x + 1 x + 11 x + 10
```

Let's take an example of:

$$A = 10x^2 + 6x + 2B = 12x^2 + 3x + 2 \quad (2.38)$$

If we add these, we get:

$$A + B = 10x^2 + 6x + 2 + 12x^2 + 3x + 2 = 22x^2 + 9x + 4 \quad (2.39)$$

Within cryptography, we normally use finite fields, where the values are constrained by 0 and $p - 1$, and where p is a prime number. For this, we can perform a $(\text{mod } p)$ operation on this, such as with $(\text{mod } 13)$:

$$A + B = 22x^2 + 9x + 4 = 9x^2 + 9x + 4 \pmod{13} \quad (2.40)$$

Now let's look at multiplication. For this, we have:

$$\begin{aligned} A \times B &= (10x^2 + 6x + 2) \times (12x^2 + 3x + 2) \\ &= 120x^4 + 30x^3 + 20x^2 + 72x^3 + 180x^2 + 12x + 24x^2 + 6x + 4 \\ &= 120x^4 + 102x^3 + 62x^2 + 18x + 4 \end{aligned}$$

And now, we can apply a $(\text{mod } 13)$ operation to each of the coefficients:

$$A \times B = 120x^4 + 102x^3 + 62x^2 + 18x + 4 = 3x^4 + 11x^3 + 10x^2 + 5x + 4 \pmod{13}$$

Next, we will perform a polynomial subtraction:

$$A - B = (10x^2 + 6x + 2) - (12x^2 + 3x + 2) = -2x^2 + 3x \quad (2.41)$$

And now we take $(\text{mod } 13)$:

$$A - B = -2x^2 + 3x = 11x^2 + 3x \pmod{13} \quad (2.42)$$

Next, we will perform a polynomial division:

$$A \div B = (10x^2 + 6x + 2)(12x^2 + 3x + 2) = 3x \quad (2.43)$$

And now $(\text{mod } 13)$:

$$A \div B \pmod{13} = 3x \quad (2.44)$$

Code to implement this is [5]:

```
import numpy as np
import sys

q=13

A=[5,4,1,11,10]
```



```

B=[3,5,2,4]

if (len(sys.argv)>1):
    A=list(eval(sys.argv[1]))
if (len(sys.argv)>2):
    B=list(eval(sys.argv[2]))

print ("A:")
print(np.poly1d(A))
print ("B:")
print(np.poly1d(B))

print ("\n\n==== A+B =====")
res=np.polyadd(A,B)
print(np.poly1d(res))

res=np.polyadd(A,B)%q

print ("\nA+B (mod",q,"):")
print(np.poly1d(res))

print ("\n\n==== A*B =====")
res=np.polymul(A,B)
print(np.poly1d(res))

res=np.polymul(A,B)%q

print ("\n\nA*B (mod",q,"):")
print(np.poly1d(res))

print ("\n\n==== A-B =====")
res=np.polysub(A,B)

print(np.poly1d(res))

res=np.polysub(A,B)%q

print ("\n\nA-B (mod",q,"):")
print(np.poly1d(res))

print ("\n\nA/B =====")

res=np.floor(np.polydiv(A,B)[0])

```

```

print("Divide:\n",np.poly1d(res))

res=np.floor(np.polydiv(A,B)[1])
print("Remainder:\n",np.poly1d(res))

res=np.floor(np.polydiv(A,B)[0])%q
print ("\n\nA div B (mod",q,"):")
print(np.poly1d(res))

print("\n\nA div B (mod {0} ):".format(q))

print(np.poly1d(res))

```

A sample run using $A = 4x^3 + x^2 + 11x + 10$ and $B = 3x^3 + 5x^2 + 2x + 4$ is:

```

A:
  3      2
4 x + 1 x + 11 x + 10
B:
  3      2
3 x + 5 x + 2 x + 4

===== A+B =====
  3      2
7 x + 6 x + 13 x + 14

A+B (mod 13 ):
  3      2
7 x + 6 x + 1

===== A*B =====
  6      5      4      3      2
12 x + 23 x + 46 x + 103 x + 76 x + 64 x + 40

A*B (mod 13 ):
  6      5      4      3      2
12 x + 10 x + 7 x + 12 x + 11 x + 12 x + 1

===== A-B =====
  3      2
1 x - 4 x + 9 x + 6

```

```

A-B (mod 13 ):
      3      2
1 x + 9 x + 9 x + 6

A/B =====
Divide:

1
Remainder:
      2
-6 x + 8 x + 4

A div B (mod 13 ):

1

```

2.3.2 Inverse polynomial mod p

With lattice methods, we use the shortest vector problem in a lattice, which has an underpinning difficulty of factorizing polynomials. The lattice vector points are represented as polynomial values. For this, we create a modulo polynomial and then generate its inverse. In this case, we will create a polynomial (f) and then find its modulo inverse (f_p), and which will result in:

$$f \cdot f_p = 1 \pmod{p} \quad (2.45)$$

Thus, if we then take a message (m) and multiply it by f and then by f_p , we should be able to recover the message. Let's take an example with $N=11$ (the highest polynomial factor), $p=31$ and where Bob picks polynomial factors (f) of:

$$f = [-1, 1, 1, 0, -1, 0, 1, 0, 0, 1, -1] \quad (2.46)$$

$$f(x) = -1x^{10} + 1x^9 + 1x^6 - 1x^4 + 1x^2 + 1x - 1 \pmod{31} \quad (2.47)$$

We then determine the inverse of this with:

$$f_p : [9, 5, 16, 3, 15, 15, 22, 19, 18, 29, 5] \quad (2.48)$$

$$f_p(x) = 5x^{10} + 29x^9 + 18x^8 + 19x^7 + 22x^6 + 15x^5 + 15x^4 + 3x^3 + 16x^2 + 5x + 9 \pmod{31} \quad (2.49)$$

So, we can now take a message (m) and multiply it by f to gain a ciphertext, and then multiply it with f_p to recover the message. In the following, we will use a message of:

$$m = [1, 0, 1, 0, 1, 1, 1, 1] \quad (2.50)$$

$$m = 1x^7 + 1x^6 + 1x^5 + 1x^4 + 1x^2 + 1 \quad (2.51)$$

A test run is [here]:

```

Values used:
N= 11
p= 31
=====
Bob picks a polynomial (f):
f(x)=  [-1, 1, 1, 0, -1, 0, 1, 0, 0, 1, -1]
10      9      6      4      2
-1 x  + 1 x  + 1 x  - 1 x  + 1 x  + 1 x  - 1
====Now we determine F_p====
F_p:  [9, 5, 16, 3, 15, 15, 22, 19, 18, 29, 5]
10      9      8      7      6      5      4      3      2
5 x  + 29 x  + 18 x  + 19 x  + 22 x  + 15 x  + 15 x  + 3 x  + 16 x  +
5 x  + 9
====Now we determine the message====
Alice's Message:  [1, 0, 1, 0, 1, 1, 1, 1]
7      6      5      4      2
1 x  + 1 x  + 1 x  + 1 x  + 1 x  + 1
====Encrypted message====
Encrypted message:  [0, 1, 2, 1, 30, 0, 0, 1, 2, 1, 30]
10      9      8      7      4      3      2
30 x  + 1 x  + 2 x  + 1 x  + 30 x  + 1 x  + 2 x  + 1 x
====Decrypted message====
Decrypted message:  [1, 0, 1, 0, 1, 1, 1, 1]
7      6      5      4      2
1 x  + 1 x  + 1 x  + 1 x  + 1 x  + 1

```

The code is based on this code [6]:

```

from fracModulo import extEuclidPoly, modPoly, multPoly,
                        reModulo

import numpy
import sys

```

```

N=11
p=31

f=[-1,1,1,0,-1,0,1,0,0,1,-1]
m=[1,0,1,0,1,1,1,1]

if (len(sys.argv)>1):
    N=int(sys.argv[1])
if (len(sys.argv)>2):
    p=int(sys.argv[2])
if (len(sys.argv)>3):
    f=eval("["+sys.argv[3]+"]")
if (len(sys.argv)>4):
    m=eval("["+sys.argv[4]+"]")

print("Values used:")
print(" N=",N)
print(" p=",p)
print("=====")
print("\nBob picks a polynomials (f):")

print("f(x)= ",f)
print (" \n",numpy.poly1d(f[::-1]))

D=[0]*(N+1)
D[0]=-1
D[N]=1

print("\n====Now we determine F_p ===")
[gcd_f,s_f,t_f]=extEuclidPoly(f,D)

f_p=modPoly(s_f,p)

print("F_p:",f_p)
print (" \n",numpy.poly1d(f_p[::-1]))

x=multPoly(f,m)
enc=reModulo(x,D,p)

x=multPoly(enc,f_p)
dec=reModulo(x,D,p)[:len(m)]

print("\n====Now we determine F_p ===")
print("Alice's Message:\t",m)
print (" \n",numpy.poly1d(m[::-1]))

```

```

print("\n===Encrypted message ===")
print("Encrypted message:\t",enc)
print ("\n",numpy.poly1d(enc[::-1]))

print("\n===Decrypted message ===")
print("Decrypted message:\t",dec)
print ("\n",numpy.poly1d(dec[::-1]))

```

2.4 Learning With Errors

Learning With Errors (LWE) is a quantum robust method of cryptography. Initially we create a secret key value (s) and another value (e). Next we select a number of values ($A[]$) and calculate $B[] = A[] \times s + e$. The values of $A[]$ and $B[]$ become our public key. If s is a single value, A and B are one dimensional matrices. If we select s to be a one-dimensional matrix, A will be a two-dimensional matrix, and B will be a one-dimensional matrix.

Learning with errors is a method defined by Oded Regev in 2005 [7] and is known as LWE (Learning With Errors). It involves the difficulty of finding the values which solve:

$$\mathbf{B} = \mathbf{A} \times \mathbf{s} + \mathbf{e} \quad (2.52)$$

where you know $\mathbf{A}$ and $\mathbf{B}$. The value of $\mathbf{s}$ becomes the secret values (or the secret key), and $\mathbf{A}$ and $\mathbf{B}$ can become the public key. A video is here [8].

2.4.1 Basics for LWE

In this presentation we will use the examples defined at [9]. The following code implements a basic LWE method:

```

import numpy as np
import sys

q=13

A=np.array([[4 ,1, 11, 10],[5, 5 ,9 ,5],[3, 9 ,0 ,10],[1, 3 ,
              3 ,2],[12, 7 ,3 ,4],[6, 5 ,11
              ,4],[3, 3, 5, 0]])

sA = np.array([[6],[9],[11],[11]])
eA = np.array([[0],[-1],[1],[1],[1],[0],[-1]])

bA = np.matmul(A,sA)\%q
print (bA)

```

```

bA = np.add(bA,eA)\%q
print
print ("Print output\n",bA)

```

$$b_A = \begin{bmatrix} 4 & 1 & 11 & 10 \\ 5 & 5 & 9 & 5 \\ 3 & 9 & 0 & 10 \\ 1 & 3 & 3 & 2 \\ 12 & 7 & 3 & 4 \\ 6 & 5 & 11 & 4 \\ 3 & 3 & 5 & 0 \end{bmatrix} \times \begin{bmatrix} 6 \\ 9 \\ 11 \\ 11 \end{bmatrix} + \begin{bmatrix} 0 \\ -1 \\ 1 \\ 1 \\ 1 \\ 0 \\ -1 \end{bmatrix} \pmod{13} = \begin{bmatrix} 4 \\ 7 \\ 2 \\ 11 \\ 5 \\ 12 \\ 8 \end{bmatrix}$$

2.4.2 Ring LWE

In this presentation we will use the examples defined at [9]. The following code implements a basic Ring LWE method:

```

import numpy as np

A = [4,1,11,10]
sA = [6,9,11,11]
eA =[0,-1,1,1]

n=len(A)
q=13

print (A,sA,eA)

xN_1 = [1] + [0] * (n-1) + [1]

print (xN_1)
A = np.floor(np.polydiv(A,xN_1)[1])

bA = np.polymul(A,sA)%q
bA = np.floor(np.polydiv(bA,xN_1)[1])

bA = np.polyadd(bA,eA)%q
bA = np.floor(np.polydiv(bA,xN_1)[1])
print ("Print output\n",bA)

```

In this method we perform a similar method to the Diffie Hellman method, but use Ring Learning With Errors (RLWE). With RLWE we use the coefficients of polynomials and which can be added and multiplied within a finite field ($\mathbf{F}_q$) [10] and where all the coefficients will be less than q . Initially Alice and Bob agree on a complexity value of n , and which is the highest

co-efficient power to be used. They then generate q which is $2^n - 1$. All the polynomial operations will then be conducted with a modulus of q and where the largest coefficient value will be $q - 1$. She then creates $a_i(x)$ which is a set of polynomial values:

$$\mathbf{A} = a_{n-1}x^{n-1} + \dots + a_1x + a_1x^2 + a_0 \quad (2.53)$$

Next Alice will divide by $\Phi(x)$, which is $x^n + 1$:

$$\mathbf{A} = (a_{n-1}x^{n-1} + \dots + a_1x + a_1x^2 + a_0) \div (x^n + 1) \quad (2.54)$$

In Python this is achieved with:

```
xN_1 = [1] + [0] * (n-1) + [1]
A = np.floor(p.polydiv(A, xN_1)[1])
```

$$e_A = e_{n-1}x^{n-1} + \dots e_2x^2 + e_1x + e_0 \pmod{q} \quad (2.55)$$

$$s_A = s_{n-1}x^{n-1} + \dots s_2x^2 + s_1x + s_0 \pmod{q} \quad (2.56)$$

Alice now creates b_A which is a polynomial created from A , e_A and s_A :

$$b_A = A \times s_A + e_A \quad (2.57)$$

In coding this is defined as:

```
bA = p.polymul(A, sA) \% q
bA = np.floor(p.polydiv(sA, xN_1)[1])
bA = p.polyadd(bA, eA) \% q
```

Bob now generates his own error polynomial e' and a secret polynomial s' :

$$e_B = e'_{n-1}x^{n-1} + \dots e'_2x^2 + e'_1x + e'_0 \pmod{q} \quad (2.58)$$

$$s_B = s'_{n-1}x^{n-1} + \dots s'_2x^2 + s'_1x + s'_0 \pmod{q} \quad (2.59)$$

Alice shares A with Bob, and Bob now creates b_B which is a polynomial created from A , e_B and s_B :

$$b_B = A \times s_B + e_B \quad (2.60)$$

The code for this is:


```

sB = gen_poly(n,q)
eB = gen_poly(n,q)

bB = p.polymul(A,sB)\%q
bB = np.floor(p.polydiv(sB,xN_1)[1])
bB = p.polyadd(bB,eB)\%q

```

Alice then takes Bob's value (b_B) and multiplies by s_A and divides by $x^n + 1$:

$$shared_A = b_B \times s_A \div (x^n + 1) \quad (2.61)$$

Bob then takes Alice's value (b_A) and multiplies by s_B and divides by $x^n + 1$:

$$shared_B = b_A \times s_B \div (x^n + 1) \quad (2.62)$$

At the end of this, Bob and Alice will have the same shared value (and thus can create a shared key based on it).

In code this is:

```

sharedAlice = np.floor(p.polymul(sA,bB) \%q)
sharedAlice = np.floor(p.polydiv(sharedAlice,xN_1)[1])\%q
sharedBob = np.floor(p.polymul(sB,bA)\%q)
sharedBob = np.floor(p.polydiv(sharedBob,xN_1)[1])\%q

```

Chapter 3

State-of-the-art

3.1 Introduction

Homomorphic encryption supports mathematical operations on encrypted data. In 1978, Rivest, Adleman, and Dertouzos [11] were the first to define the possibilities of implementing a homomorphic operation and used the RSA method. This supported multiply and divide operations [12], but does not support addition and subtraction. Overall, Partial Homomorphic Encryption (PHE) supports a few arithmetic operations, while Fully Homomorphic Encryption (FHE) supports add, subtract, multiply, and divide.

Since, Gentry defined the first FHE method [13] in 2009, there have been four main generations of homomorphic encryption:

- 1st generation: Gentry's method uses integers and lattices [14] including the DGHV method.
- 2nd generation. Brakerski, Gentry and Vaikuntanathan's (BGV) and Brakerski/Fan-Vercauteren (BFV) use a Ring Learning With Errors approach [15]. The methods are similar to each other, and with only small difference between them.
- 3rd generation: These include DM (also known as FHEW) and CGGI (also known as TFHE) and support the integration of Boolean circuits for small integers.
- 4th generation: CKKS (Cheon, Kim, Kim, Song) and which uses floating-point numbers [16].

Generally, CKKS works best for real number computations and can be applied to machine learning applications as it can implement logistic regression

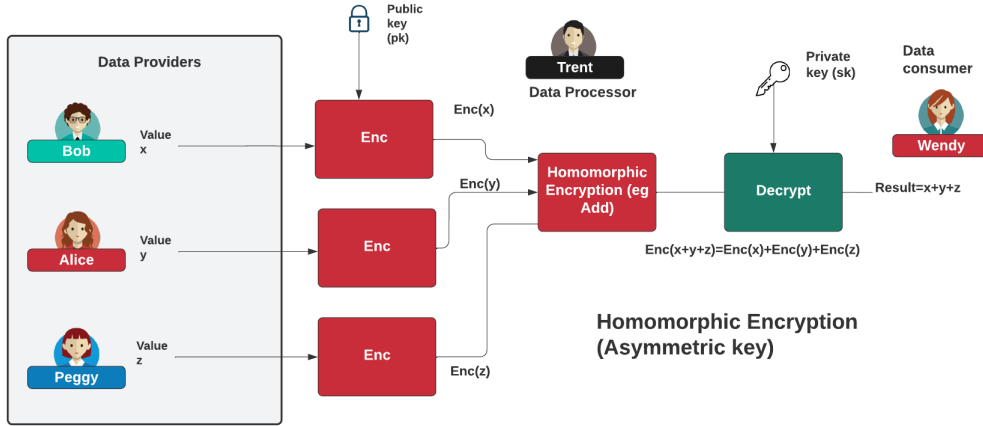


Figure 3.1: Asymmetric encryption (public key)

methods and other statistical computations. DM (also known as FHEW) and CGGI (also known as TFHE) are useful in the application of Boolean circuits for small integers. BGV and BFV are generally used in applications with small integer values.

3.2 Public key or symmetric key

Homomorphic encryption can be implemented either with a symmetric key or an asymmetric (public) key. With symmetric key encryption, we use the same key to encrypt as we do to decrypt, whereas, with an asymmetric method, we use a public key to encrypt and a private key to decrypt. In Figure 3.1 we use asymmetric encryption with a public key (pk) and a private key (sk). With this Bob, Alice and Peggy will encrypt their data using the public key to produce ciphertext, and then we can operate on the ciphertext using arithmetic operations. The result can then be revealed by decrypting with the associated private key. In Figure 3.2 we use symmetric key encryption, and where the data is encrypted with a secret key, and which is then used to decrypt the data. In this case, the data processor (Trent) should not have access to the secret key, as they could decrypt the data from the providers.

3.3 Homomorphic libraries

There are several homomorphic encryption libraries that support FHe, including ones which support CUDA and GPU acceleration, but many have not

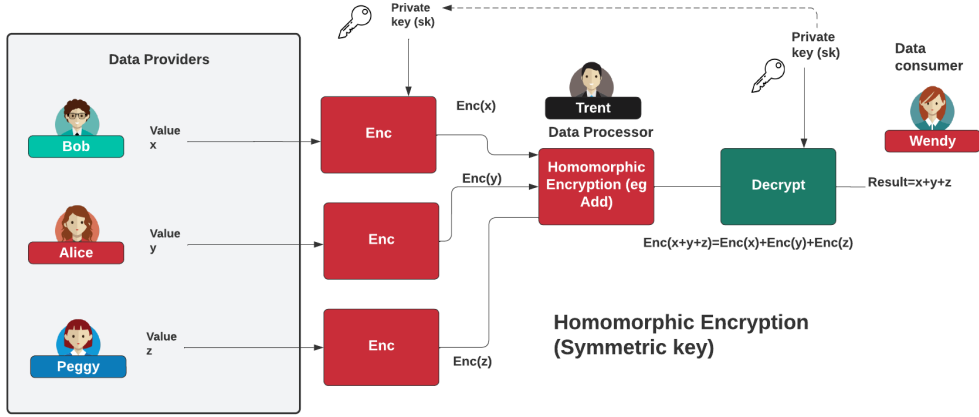


Figure 3.2: Symmetric encryption

been kept up-to-date with modern methods, or only integrate one method. Overall, the native language libraries tend to be the most useful, as they allow the compilation to machine code. The main language for this are C++, Golang and Rust, although some Python libraries exist through wrappers to C++ code. This includes HEAAN-Python, and its associated HEAAN library.

One of the first libraries which supported a range of methods are Microsoft SEAL [17], SEAL-C# [18] and SEAL-Python. While it supports a wide range of methods, including BGV/BFV and CKKS, it has lack any real serious development for the past few years. It does have support for Android and has a Node.js port [19]. One of the most extensive libraries is PALISADE, and which has now developed into OpenFHE. Within OpenFHE, the main implementations are:

- Brakerski/Fan-Vercauteren (**BFV**) scheme for integer arithmetic
- Brakerski-Gentry-Vaikuntanathan (**BGV**) scheme for integer arithmetic
- Cheon-Kim-Kim-Song (**CKKS**) scheme for real-number arithmetic (includes approximate bootstrapping)
- Ducas-Micciancio (**DM**) and Chillotti-Gama-Georgieva-Izabachene (**CGGI**) schemes for Boolean circuit evaluation.

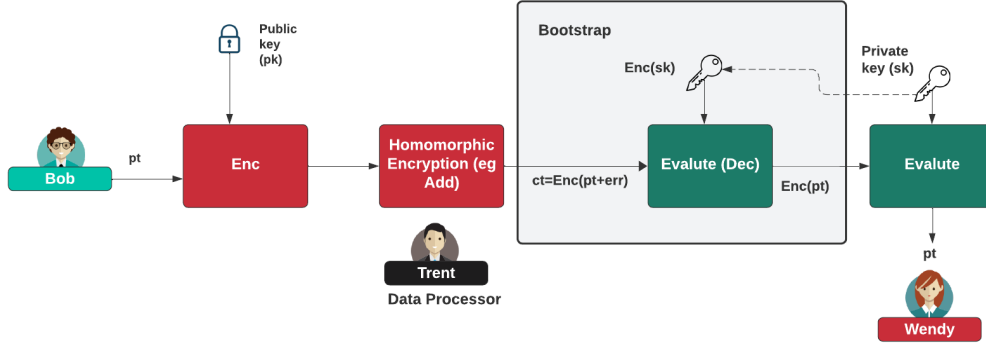


Figure 3.3: Bootstrap

3.4 Bootstrapping

A key topic within fully homomorphic encryption is the usage of bootstrapping. Within a learning with-errors approach, we add noise to our computations. For a normal decryption process, we use the public key to encrypt data and then the associated private key to decrypt it. Within the bootstrap version of homomorphic encryption, we use an encrypted version of the private key that operates on the ciphertext. In this way, we remove the noise which can build up in the computation. Figure 3.3 outlines that we perform and evaluation on the decryption using an encrypted version of the private key. This will remove noise in the ciphertext, after which we can then use the actual private key to perform the decryption.

The main bootstrapping methods are CKKS [16], DM [20]/CGGI, and BGV/BFV. Overall, CKKS is generally the fastest bootstrapping method, while DM/CGGI is efficient with the evaluation of arbitrary functions. These functions approximate maths functions as polynomials (such as with Chebyshev approximation). BGV/BFV provides reasonable performance and is generally faster than DM/CGGI but slower than CKKS.

3.5 Arbitrary smooth functions

With approximation theory, it is possible to determine an approximate polynomial $p(x)$ that is approximate to a function $f(x)$. A polynomial takes the form of $p(x) = a_n \cdot x^n + a_{n-1} \cdot x^{n-1} + \dots + a_1 \cdot x + a_0$, and where $a_0 \dots a_n$ are the coefficients of the powers, and n is the maximum power of the polynomial.

For this, we can define arbitrary smooth functions for CKKS using Chebyshev approximation [21]. These were initially created by Pafnuty Lvovich

Chebyshev. This method involves the approximation of a smooth function using polynomials. Examples of these functions include $\log_{10}$, $\log_2$, $\log_e$, and e^x [22].

3.6 Plaintext slots

With many of homomorphic method, we can encrypt multiple plaintext values into ciphertext in a single operation. This is defined as the number of plaintext slots.

3.7 DGHV

Craig Gentry was the first to show an implementation of fully homomorphic encryption (FHE), and received the 2009 ACM Doctoral Dissertation Award for this breakthrough work. The concept of homomorphic encryption, though, had been around since public key encryption itself, and partial homomorphic methods included Paillier [1] and RSA [4]. Overall, homomorphic encryption can either be partially homomorphic (PHE), somewhat homomorphic, levelled fully homomorphic, or **fully homomorphic encryption** (FHE). For security, the best case is fully homomorphic encryption. Craig's published work on homomorphic encryption started in 2019 with the first useable fully homomorphic encryption (FHE) method [23].

The work of Gentry was then advanced, and, in 2010, DGHV (Dijk, Gentry, Halevi and Vaikuntanathan) defined an expansion into the usage of integers [14]. With these methods, we encrypt our data and then input these cipher bits into a cipher circuit. A basic adding function is here [24]. Overall it was not possible to examine any of the bits in the circuit to discover the original state of the plaintext. On the output of the circuit, we could then decrypt the cipher bits back into the cipher text. For this, we could create the function or arithmetic operation that we wanted ... it all comes down to EX-OR (adding) and AND (multiplication) operations.

3.8 BGV and BFV

With BGV and BFV, we use a Ring Learning With Errors (LWE) method [15]. With BGV, we define a moduli (q), which constrains the range of the polynomial coefficients. Overall, the methods use a moduli, which can be defined within different levels. We then initially define a finite group of $\mathbb{Z}_q$, and then make this a ring by dividing our operations with $(x^n + 1)$ and

where $n - 1$ is the largest power of the coefficients. The message can then be represented in binary as:

$$m = a_{n-1}a_{n-2}\dots a_0 \quad (3.1)$$

This can be converted into a polynomial with:

$$\mathbf{m} = a_{n-1}x^n a_{n-1} + a_{n-2}x^{n-2} + \dots + a_1x + a_0 \pmod{q} \quad (3.2)$$

The coefficients of this polynomial will then be a vector. Note that for efficiency, we can also encode the message with ternary (such as with -1, 0 and 1). We then define the plaintext modulus with:

$$t = p^r \quad (3.3)$$

and where p is a prime number and r is a positive number. We can then define a ciphertext modulus of q , and which should be much larger than t . To encrypt with the private key of $\mathbf{s}$, we implement:

$$(c_0, c_1) = \left(\frac{q}{t} \cdot \mathbf{m} + \mathbf{a} \cdot \mathbf{s} + e, -\mathbf{a} \right) \pmod{q} \quad (3.4)$$

To decrypt:

$$m = \left\lfloor \frac{t}{q} (c_0 + c_1) \cdot \mathbf{s} \right\rfloor \quad (3.5)$$

This works because:

$$m_{recover} = \left\lfloor \frac{t}{q} \left(\frac{q}{t} \cdot \mathbf{m} + \mathbf{a} \cdot \mathbf{s} + e - \mathbf{a} \cdot \mathbf{s} \right) \right\rfloor \quad (3.6)$$

$$= \left\lfloor \left(\mathbf{m} + \frac{t}{q} \cdot e \right) \right\rfloor \quad (3.7)$$

$$\approx m \quad (3.8)$$

$$(3.9)$$

For two message of m_1 and m_2 , we will get:

$$Enc(m_1 + m_2) = Enc(m_1) + Enc(m_2) \quad (3.10)$$

$$Enc(m_1 \cdot m_2) = Enc(m_1) \cdot Enc(m_2) \quad (3.11)$$

3.8.1 Noise and computation

But each time we add or multiply, the error also increases. Thus bootstrapping is required to reduce the noise. Overall, addition and plaintext/ciphertext multiplication is not a time-consuming task, but ciphertext/ciphertext multiplication is more computationally intensive. The most computational task is typically the bootstrapping process, and the ciphertext/ciphertext multiplication process adds the most noise to the process.

3.8.2 Parameters

We thus have a parameter of the ciphertext modulus (q) and the plaintext modulus (t). Both of these are typically to the power of 2. An example of q is 2^{240} and for t is 65,537. As the value of 2^q is likely to be a large number, we typically define it as a $\log_q$ value. Thus a ciphertext modulus of 2^{240} will be 240 as defined as a $\log_q$ value.

3.8.3 Public key generation

We select the private (secret) key using a random ternary polynomial (-1, 0, and 0 coefficients) and which has the same degree as our ring. The public key is then a pair of polynomials as:

$$\mathbf{pk}_1 = (\mathbf{r.sk} + e) \pmod{q} \quad (3.12)$$

$$\mathbf{pk}_2 = \mathbf{r} \quad (3.13)$$

and where r is a random polynomial value. To encrypt with the public key (pk):

$$(c_0, c_1) = \left(\frac{q}{t} \cdot \mathbf{m} + \mathbf{a.s.r} + e, -\mathbf{a.r} \right) \pmod{q} \quad (3.14)$$

We then decrypt with the private key (s);

$$m = \left\lfloor \frac{t}{q} (c_0 + c_1) \cdot \mathbf{s} \right\rfloor \quad (3.15)$$

This works because:

$$m_{recovered} = \lfloor \frac{t}{q} \left(\frac{q}{t} \cdot \mathbf{m} + \mathbf{a.s.r} + e - \mathbf{a.r.s.} \right) \rfloor \quad (3.16)$$

$$= \lfloor \left(\mathbf{m} + \frac{t}{q} \cdot e \right) \rfloor \quad (3.17)$$

$$\approx m \quad (3.18)$$

$$(3.19)$$

3.9 DM/FHEW and CGGI/TFHE

The work in the 3rd generation of the method outlined the FHEW scheme [25] that allowed for the refreshing of ciphertexts after every operation. This reduced the bootstrapping time to less than a second. It can be used with the bootstrapping technique proposed by Ducas and Micciancio (DM) [20]. Overall, FHEW uses Boolean gates applied to cipher data bits, and which simplifies the bootstrapping process. An enhancement for DM(FHEW) is to use the CGGI(TFHE) schemes [26].

3.9.1 DM

DM (FHEW) [20] uses an LWE (Learning With Error) method that provides fully homomorphic encryption (FHE). It is able to evaluate cipher data applied onto Boolean circuits and uses bootstrapping after each gate evaluation. This allows the evaluation to be conducted in less than 0.1 seconds. The gates implemented are AND, OR, NAND, NOR, and NOT. In homomorphic encryption, the noise in the computation increases as we perform operations. This is typically when we perform a large number of additions. and multiplications will often reduce the amount of noise. A bootstrapping process allows for us to reset the noise and will introduce a delay in the computation. The OpenFHE library supports the usage of the GINX bootstrapping method [27], AP [28], or LMKCDEY [29].

3.10 CKKS

HEAAN (Homomorphic Encryption for Arithmetic of Approximate Numbers) defines a homomorphic encryption (HE) library proposed by Cheon, Kim, Kim and Song (CKKS). The CKKS method uses approximate arithmetics over complex numbers [16]. Overall, it is a levelled approach that in-

volves the evaluation of arbitrary circuits of bounded (pre-determined) depth. These circuits can include ADD (X-OR) and Multiply (AND).

HEAAN uses a rescaling procedure to measure the size of the plaintext. It then produces an approximate rounding due to the truncation of the ciphertext into a smaller modulus. The method is especially useful in that it can be applied to carry out encryption computations in parallel. Unfortunately, the ciphertext modulus can become too small, and where it is not possible to carry out any more operations.

The HEAAN (CKKS) method uses approximate arithmetic over complex numbers ($\mathbb{C}$), and is based on Ring Learning With Errors (RLWE). It focuses on defining an encryption error within the computational error that will happen within approximate computations. We initially take a message (M) and convert it to a cipher message (ct) using a secret key sk . To decrypt ($[[ct, sk]]_q$), we produce an approximate value along with a small error (e).

Craig Gentry [23] has outlined three important application areas within privacy-preserving genome association, neural networks, and private information retrieval. Along with this, he proposed that the research community should investigate new methods which **did not involve the usage of lattices**.

3.11 Enhanced features

Homomorphic encryption support a wide range of feature, including that it is Post Quantum robust. This section outlines some of the additional features that can be used with homomomorphic encryption.

3.11.1 Proxy re-encryption

With public key encryption, we can encrypt with a public key and then decrypt with a private key. And, so, if we have some ciphertext, and we want to pass this onto Bob, we would have to re-encrypt by decrypting with Alice's private key, and then re-encrypt with Bob's public key. With proxy re-encryption (PRE), we can convert ciphertext encrypted with Alice's public key into one that Bob can decrypt with his private key. as shown in Figure 3.4. This feature is supported in OpenFHE [30].

In this case, we might have a trusted proxy agent, such as Trent, and who is trusted to use Alice's private key. In this case, we might have a trusted proxy agent, such as Trent, and who is trusted to use Alice's private key. Trent can either be trusted to hold Alice's private key, or Alice can use her private key and Bob's public key to create the re-encryption key.

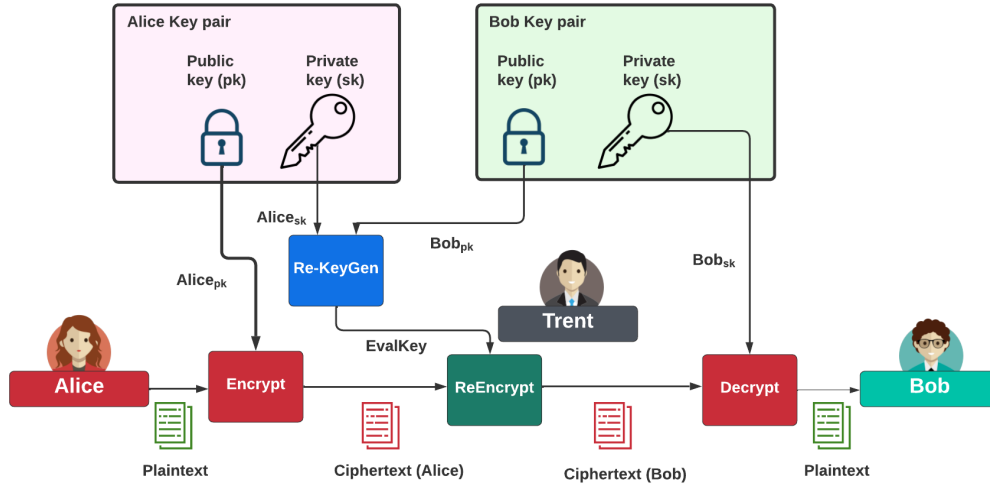


Figure 3.4: Proxy re-encryption

3.11.2 Threshold encryption

With MPC (Multiparty Computation) we can split a task into a number of secure computations. In this case, we will use threshold encryption to split the decryption process into three encryption key shares. Overall, we will compute the adding of three data sets using homomorphic encryption, and then allow Bob, Alice and Carol to generate decrypted values, and then which can be aggregated together to provide the result. This is illustrated in Figure 3.5 and implemented in OpenFHE [31].

3.11.3 Machine learning

Homomorphic encryption supports the usage machine learning methods, and some core features include a dot product operation with an encrypted vector and logistic functions. With this, OpenFHE supports a range of relevant methods, and even has a demonstrator for a machine learning method.

3.12 Conclusions

For the state-of-the-art, the best library for support seems to be OpenFHE, as it supports BGV/BFV, CKKS and DM/CGGI, along with other cryptographic primitives, such as proxy re-encryption and secret shares. Overall,

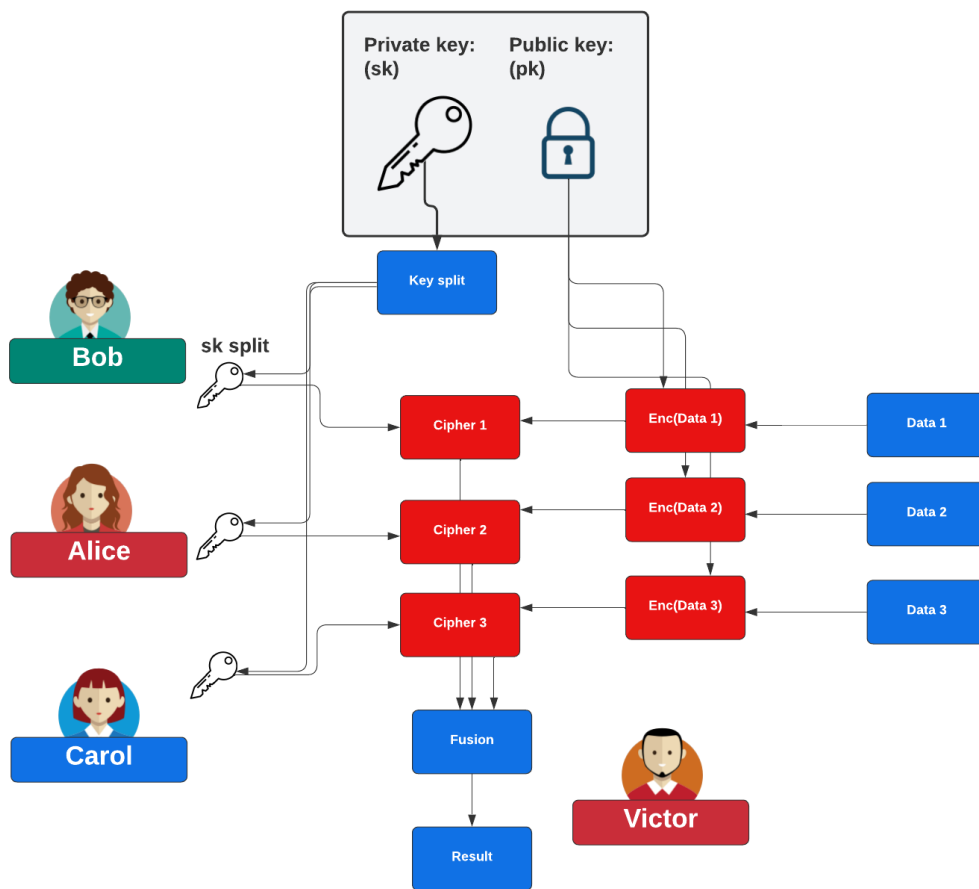


Figure 3.5: Threshold encryption

CKKS is the fastest method for homomomorphic encryption, and support floating point values. The DM/CGGI method support Boolean logic circuit, but can be slow in the computation. For performance, the bootstrapping processing and ciphertext-ciphertext multiplication has the highest performance impact.

Bibliography

- [1] P. Paillier, “Public-key cryptosystems based on composite degree residuosity classes,” in *International conference on the theory and applications of cryptographic techniques*. Springer, 1999, pp. 223–238.
- [2] W. J. Buchanan, “Paillier - partial homomorphic encryption,” <https://asecuritysite.com/paillier/>, Asecuritysite.com, 2024, accessed: September 03, 2024. [Online]. Available: <https://asecuritysite.com/paillier/>
- [3] —, “Partial homomorphic encryption with elgamal (multiply),” https://asecuritysite.com/homomorphic/go_el_homo, Asecuritysite.com, 2024, accessed: September 03, 2024. [Online]. Available: https://asecuritysite.com/homomorphic/go_el_homo
- [4] —, “Multiplication with homomorphic encryption using rsa,” https://asecuritysite.com/homomorphic/hom_rsa, Asecuritysite.com, 2024, accessed: September 03, 2024. [Online]. Available: https://asecuritysite.com/homomorphic/hom_rsa
- [5] —, “Polynomials operations (mod p),” <https://asecuritysite.com/encryption/poly4>, Asecuritysite.com, 2024, accessed: September 03, 2024. [Online]. Available: <https://asecuritysite.com/encryption/poly4>
- [6] —, “Multiplying by the inverse of a polynomial,” https://asecuritysite.com/principles/poly_2, Asecuritysite.com, 2024, accessed: September 03, 2024. [Online]. Available: https://asecuritysite.com/principles/poly_2
- [7] O. Regev, “On lattices, learning with errors, random linear codes, and cryptography,” *Journal of the ACM (JACM)*, vol. 56, no. 6, pp. 1–40, 2009.
- [8] W. J. Buchanan, “Learning with errors,” <https://www.youtube.com/embed/hN5TQiz2gWs>, Asecuritysite.com, 2024, accessed: September

- 03, 2024. [Online]. Available: <https://www.youtube.com/embed/hN5TQiz2gWs>
- [9] D. Stebila, “Introduction to post-quantum cryptography and learning with errors,” <https://summerschool-croatia.cs.ru.nl/2018/slides/Introduction%20to%20post-quantum%20cryptography%20and%20learning%20with%20errors.pdf>, Asecuritysite.com, 2018, accessed: September 03, 2024. [Online]. Available: <https://summerschool-croatia.cs.ru.nl/2018/slides/Introduction%20to%20post-quantum%20cryptography%20and%20learning%20with%20errors.pdf>
 - [10] W. J. Buchanan, “Finite fields,” <https://asecuritysite.com/gf/finite>, Asecuritysite.com, 2024, accessed: September 03, 2024. [Online]. Available: <https://asecuritysite.com/gf/finite>
 - [11] R. L. Rivest, L. Adleman, M. L. Dertouzos *et al.*, “On data banks and privacy homomorphisms,” *Foundations of secure computation*, vol. 4, no. 11, pp. 169–180, 1978.
 - [12] W. J. Buchanan, “Rsa - partially homomorphic cryptosystem: Division,” https://asecuritysite.com/homomorphic/h_rsa2, Asecuritysite.com, 2024, accessed: September 04, 2024. [Online]. Available: https://asecuritysite.com/homomorphic/h_rsa2
 - [13] C. Gentry, “A fully homomorphic encryption scheme,” 2009, crypto.stanford.edu/craig.
 - [14] M. Van Dijk, C. Gentry, S. Halevi, and V. Vaikuntanathan, “Fully homomorphic encryption over the integers,” in *Advances in Cryptology—EUROCRYPT 2010: 29th Annual International Conference on the Theory and Applications of Cryptographic Techniques, French Riviera, May 30–June 3, 2010. Proceedings 29*. Springer, 2010, pp. 24–43.
 - [15] Z. Brakerski and V. Vaikuntanathan, “Efficient fully homomorphic encryption from (standard) lwe,” *SIAM Journal on computing*, vol. 43, no. 2, pp. 831–871, 2014.
 - [16] J. H. Cheon, A. Kim, M. Kim, and Y. Song, “Homomorphic encryption for arithmetic of approximate numbers,” in *Advances in Cryptology—ASIACRYPT 2017: 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3–7, 2017, Proceedings, Part I 23*. Springer, 2017, pp. 409–437.

- [17] W. J. Buchanan, “Homomorphic encryption (seal),” <https://asecuritysite.com/seal>, Asecuritysite.com, 2024, accessed: September 04, 2024. [Online]. Available: <https://asecuritysite.com/seal>
- [18] —, “Encrypting integers with bgv and bfv using seal and c#,” https://asecuritysite.com/seal/seal_01, Asecuritysite.com, 2024, accessed: September 04, 2024. [Online]. Available: https://asecuritysite.com/seal/seal_01
- [19] —, “Homomorphic encryption with bfv using node.js,” https://asecuritysite.com/seal/js_homomorphic, Asecuritysite.com, 2024, accessed: September 04, 2024. [Online]. Available: https://asecuritysite.com/seal/js_homomorphic
- [20] L. Ducas and D. Micciancio, “Fhe: bootstrapping homomorphic encryption in less than a second,” in *Annual international conference on the theory and applications of cryptographic techniques*. Springer, 2015, pp. 617–640.
- [21] A. Al Badawi and Y. Polyakov, “Demystifying bootstrapping in fully homomorphic encryption,” *Cryptology ePrint Archive*, 2023.
- [22] W. J. Buchanan, “Chebyshev approximations using openfhe and c++ (logarithm methods),” https://asecuritysite.com/openfhe/openfhe_18cpp, Asecuritysite.com, 2024, accessed: September 04, 2024. [Online]. Available: https://asecuritysite.com/openfhe/openfhe_18cpp
- [23] C. Gentry, “Fully homomorphic encryption using ideal lattices,” in *Proceedings of the forty-first annual ACM symposium on Theory of computing*, 2009, pp. 169–178.
- [24] W. J. Buchanan, “Simple homomorphic cipher for 2-bit adder,” https://asecuritysite.com/homomorphic/hom_adder, Asecuritysite.com, 2024, accessed: September 03, 2024. [Online]. Available: https://asecuritysite.com/homomorphic/hom_adder
- [25] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachene, “Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds,” in *Advances in Cryptology–ASIACRYPT 2016: 22nd International Conference on the Theory and Application of Cryptology and Information Security, Hanoi, Vietnam, December 4–8, 2016, Proceedings, Part I 22*. Springer, 2016, pp. 3–33.

- [26] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, “Tfhe: fast fully homomorphic encryption over the torus,” *Journal of Cryptology*, vol. 33, no. 1, pp. 34–91, 2020.
- [27] N. Gama, M. Izabachène, P. Q. Nguyen, and X. Xie, “Structural lattice reduction: generalized worst-case to average-case reductions and homomorphic cryptosystems,” in *Advances in Cryptology–EUROCRYPT 2016: 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Vienna, Austria, May 8-12, 2016, Proceedings, Part II 35*. Springer, 2016, pp. 528–558.
- [28] J. Alperin-Sheriff and C. Peikert, “Faster bootstrapping with polynomial error,” in *Advances in Cryptology–CRYPTO 2014: 34th Annual Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2014, Proceedings, Part I 34*. Springer, 2014, pp. 297–314.
- [29] Y. Lee, D. Micciancio, A. Kim, R. Choi, M. Deryabin, J. Eom, and D. Yoo, “Efficient fhe bootstrapping with small evaluation keys, and applications to threshold homomorphic encryption,” in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 2023, pp. 227–256.
- [30] W. J. Buchanan, “Proxy re-encryption (pre) with ckks homomorphic encryption using openfhe and c++,” https://asecuritysite.com/openfhe/openfhe_21cpp, Asecuritysite.com, 2024, accessed: September 04, 2024. [Online]. Available: https://asecuritysite.com/openfhe/openfhe_21cpp
- [31] —, “Bfv with threshold encryption using openfhe and c++,” https://asecuritysite.com/openfhe/openfhe_16cpp, Asecuritysite.com, 2024, accessed: September 04, 2024. [Online]. Available: https://asecuritysite.com/openfhe/openfhe_16cpp